

Module 4 – Introduction to DBMS

1. Introduction to SQL

1. What is SQL, and why is it essential in database management?

SQL (Structured Query Language) is a language used to **talk to databases**. It helps you **store, find, change, and delete data** easily.

Why SQL is Important:

1. **Manage data:** Helps organize and control large amounts of data.
2. **Find data:** You can search for the exact information you need.
3. **Change data:** You can add, edit, or remove data anytime.
4. **Create databases:** You can make new tables and design how data is stored.
5. **Keep data safe:** You can give permission to only some people to use or change data.
6. **Works everywhere:** SQL works in almost all database systems like MySQL, Oracle, and SQL Server.

2. Explain the difference between DBMS and RDBMS.

- ☐ DBMS just stores data.
- ☐ RDBMS stores data **in tables** and **connects them together** using relationships.

Feature	DBMS (Database Management System)	RDBMS (Relational Database Management System)
Meaning	It is software that stores and manages data.	It is software that stores data in tables (rows and columns) and manages the relationships between them.
Data Storage	Data is stored as files.	Data is stored in tables.
Relationship Between Data	No relationship between data.	Maintains relationships using foreign keys .
Data Redundancy	High (same data may be repeated).	Low (reduces repetition using relations).
Data Access	Uses file system to access data.	Uses SQL to access data.
Examples	dBase, File System	MySQL, Oracle, SQL Server, PostgreSQL

3. Describe the role of SQL in managing relational databases.

SQL (Structured Query Language) is a special language used to **manage data** stored in **relational databases**.

Think of SQL like a **remote control** that helps you control the database. Here is

what SQL helps you do:

1. Store Data

SQL helps you **add new data** into tables. (Like adding a new student's record)

2. Read Data

SQL helps you **find or view data** from the database. (Like searching for a student by name)

3. Update Data

SQL helps you **change/update data** in the database. (Like correcting a student's age)

4. Delete Data

SQL helps you **remove data** you don't need. (Like removing a student who left school)

5. Create Tables

SQL helps you **build new tables** to store different types of data.

6. Control Access

SQL helps you decide **who can access or change data** — this keeps your data safe.

In short (very simple):

SQL is used to store, search, update, and manage data in relational databases. It makes database work easy, fast, and safe.

4. What are the key features of SQL?

SQL (Structured Query Language) is a powerful language used to manage and work with data stored in relational databases. It provides many important features that make data handling easy, fast, and reliable. The key features of SQL are:

1. Easy to Learn and Use

SQL uses simple English-like keywords such as **SELECT**, **INSERT**, **UPDATE**, **DELETE**, so it is very easy to understand and write.

2. Data Definition

SQL allows you to **create**, **modify**, and **delete** database objects such as tables and views. Commands: **CREATE**, **ALTER**, **DROP**

3. Data Manipulation

SQL helps to **add**, **update**, and **delete** data stored in tables. Commands: **INSERT**, **UPDATE**, **DELETE**

4. Data Retrieval

SQL is mainly used to **retrieve or fetch data** from tables. Command: **SELECT** (the most powerful and commonly used)

5. Data Control and Security

SQL provides commands to **control user access**, set permissions, and keep data safe. Commands: **GRANT**, **REVOKE**

6. Data Integrity

SQL supports rules to maintain the **accuracy and correctness** of data. Examples:

- **PRIMARY KEY**
- **FOREIGN KEY**
- **UNIQUE**
- **NOT NULL**

These rules prevent invalid or duplicate data.

7. High Performance

SQL can handle **large amounts of data** and perform operations very fast, even on complex queries.

8. Standard Language

SQL is a **standard and widely used language** supported by all major database systems like:

- MySQL
- SQL Server
- Oracle
- PostgreSQL

So SQL works almost the same everywhere.

9. Portability

SQL can be used on different computer systems, operating systems, and devices without major changes.

10. Support for Transactions

SQL supports **transactions**, which ensure all steps of a process are completed correctly.

Commands: **COMMIT**, **ROLLBACK**

This helps keep data safe and consistent.

LAB EXERCISES:

• **Lab 1: Create a new database named school_db and a table called students with the following columns: student_id, student_name, age, class, and address.**

- CREATE DATABASE school_db;
- CREATE TABLE students(student_id int PRIMARY KEY, student_name varchar(20), age int, class varchar(100), address varchar(100));

Show query box

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.1289 seconds.)

```
CREATE TABLE students( student_id int PRIMARY KEY, student_name varchar(20), age int, class varchar(100), address varchar(100));
```

[Edit inline](#)] [Edit](#)] [Create PHP code](#)]

```
SELECT * FROM `students`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

student_id	student_name	age	class	address
------------	--------------	-----	-------	---------

Lab 2: Insert five records into the students table and retrieve all records using the SELECT statement.

Introduction to DBMS

Run SQL query/queries on table school_db.students: ?

```
1 INSERT INTO students(student_id, student_name, age, class, address)VALUES(1,'Rana
Vrujal',20,'12C','Surat');
2 INSERT INTO students(student_id, student_name, age, class, address)VALUES(2,'Amit
Jangid',22,'11A','Vadodara');
3 INSERT INTO students(student_id, student_name, age, class, address)VALUES(3,'Riya Shah', 14, '9B',
'Rajkot');
4 INSERT INTO students(student_id, student_name, age, class, address)VALUES(4,'Neha Joshi', 13, '8A',
'Rajkot');
5 INSERT INTO students(student_id, student_name, age, class, address)VALUES(5,'Karan Mehta', 15, '10B',
'Bhavnagar');
```

SELECT * FROM `students`;

	student_id	student_name	age	class	address
☐ Edit Copy Delete	1	Rana Vrujal	20	12C	Surat
☐ Edit Copy Delete	2	Amit Jangid	22	11A	Vadodara
☐ Edit Copy Delete	3	Riya Shah	14	9B	Rajkot
☐ Edit Copy Delete	4	Neha Joshi	13	8A	Rajkot
☐ Edit Copy Delete	5	Karan Mehta	15	10B	Bhavnagar

2. SQL Syntax

Theory Questions:

1. What are the basic components of SQL syntax?

SQL syntax has a few basic parts:

1. Keywords

These are special SQL words.

Example: **SELECT, INSERT, UPDATE, DELETE, FROM, WHERE**

2. Identifiers

Names of database objects. Example:

table name, column name Like:

students, age, student_name

3. Operators

Symbols used for comparison.

Example: **=, >, <, >=, <=, <>**

4. Values

The actual data you put inside the table.

Example: 'John', 20, 'India'

5. Clauses

Different parts of an SQL statement. Example:

- **SELECT** = What you want
- **FROM** = From which table
- **WHERE** = With what condition

6. Semicolon (;)

Used to end the SQL command.

Example:

SELECT * FROM students;

Very Short Easy Answer

SQL has 6 basic components:

1. Keywords
2. Identifiers
3. Operators
4. Values
5. Clauses
6. Semicolon

2. Write the general structure of an SQL SELECT statement.

Syntax: -

SELECT column1, column2, ... FROM

table_name

WHERE condition GROUP

BY column_name HAVING

condition

ORDER BY column_name ASC/DESC;

- ☐ **SELECT** → which columns you want
- ☐ **FROM** → from which table
- ☐ **WHERE** → apply a condition (optional)
- ☐ **GROUP BY** → group similar data (optional)

☐ **HAVING** → condition on grouped data (optional)

☐ **ORDER BY** → sort the result (optional)

3. Explain the role of clauses in SQL statements.

Clauses are different parts of an SQL statement.

Each clause has a special job and helps SQL understand **what you want to do**.

☒ **Important Clauses and Their Roles**

1. SELECT clause

- Tells SQL **which columns** you want.
Example: SELECT name, age

2. FROM clause

- Tells SQL **from which table** to take the data.
Example: FROM students

3. WHERE clause

- Used to **filter data** based on a condition.
Example: WHERE age > 18

4. GROUP BY clause

- Used to **group rows** that have the same values.
Example: group by class.

5. HAVING clause

- Used to apply condition **after grouping**.

6. ORDER BY clause

- Used to **sort the result** in ascending or descending order.
Example: ORDER BY age DESC

LAB EXERCISES:

Lab 1: Write SQL queries to retrieve specific columns (student_name and age) from the students table.

Query: -

```
SELECT student_name, age FROM students;
```

← T → ▼						student_name	age
☐	 Edit	 Copy	 Delete	Rana Vrujal	20		
☐	 Edit	 Copy	 Delete	Amit Jangid	22		
☐	 Edit	 Copy	 Delete	Riya Shah	14		
☐	 Edit	 Copy	 Delete	Neha Joshi	13		
☐	 Edit	 Copy	 Delete	Karan Mehta	15		
☐	 Edit	 Copy	 Delete	Raj patel	9		
☐	 Edit	 Copy	 Delete	Meet Patel	6		

Lab 2: Write SQL queries to retrieve all students whose age is greater than 10.

Query: -

```
SELECT * FROM students WHERE age>10;
```

student_id	student_name	age	class	address
1	Rana Vrujal	20	12C	Surat
2	Amit Jangid	22	11A	Vadodara
3	Riya Shah	14	9B	Rajkot
4	Neha Joshi	13	8A	Rajkot
5	Karan Mehta	15	10B	Bhavnagar

3. SQL Constraints

1. What are constraints in SQL? List and explain the different types of constraints.

Constraints are rules used in SQL to control what type of data can be stored in a table. They help keep the data correct, clean, and safe.

Types of Constraints

1. NOT NULL

The column **cannot be empty**.

2. UNIQUE

All values must be **different**. No duplicate values allowed.

3. PRIMARY KEY

A special column that is **NOT NULL + UNIQUE**. Used to identify each row.

4. FOREIGN KEY

Used to **link two tables** together.

It ensures data in both tables is connected correctly.

5. CHECK

Adds a rule to the column.

Example: age must be greater than 0.

6. DEFAULT

Gives a **default value** when no value is entered.

- Constraints = **rules for data**
- NOT NULL = cannot be empty
- UNIQUE = no duplicates
- PRIMARY KEY = unique identifier
- FOREIGN KEY = links tables
- CHECK = must follow condition
- DEFAULT = gives automatic value

2. How do PRIMARY KEY and FOREIGN KEY constraints differ?

Feature	PRIMARY KEY	FOREIGN KEY
Purpose	Uniquely identifies each row	Connects two tables
Duplicates	Not allowed	Allowed
NULL values	Not allowed	Allowed (in most cases)
Where used?	In the main table	In another table to link back
Uniqueness	Must be unique	Does not need to be unique
Relation	Does NOT create relationship	Creates relationship between tables
Example	student_id in students table	student_id in marks table

3. What is the role of NOT NULL and UNIQUE constraints?

1. NOT NULL Constraint

Role:

- Makes sure a column **cannot be empty**.
- You **must** enter a value.

Example:

A student must have a name → name cannot be empty.

2. UNIQUE Constraint

Role:

- Makes sure all values in the column are **different**.
- **No duplicate** values allowed.

Example:

Each student must have a different email → no two students can have the same email.

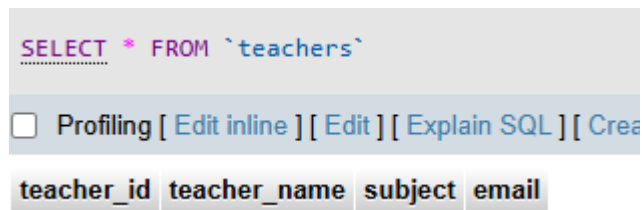
Constraint	Role
NOT NULL	Value cannot be empty
UNIQUE	No two values can be the same

LAB EXERCISES:

Lab 1: Create a table teachers with the following columns: teacher_id (Primary Key), teacher_name (NOT NULL), subject (NOT NULL), and email (UNIQUE).

Query: -

```
CREATE TABLE teachers (teacher_id int PRIMARY KEY, teacher_name varchar(50), subject VARCHAR(50), email VARCHAR(50) UNIQUE);
```



Lab 2: Implement a FOREIGN KEY constraint to relate the teacher_id from the teachers table with the students table.

Query: -

```
ALTER TABLE students ADD teacher_id INT;
```

```
ALTER TABLE students ADD CONSTRAINT fk_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id);
```

←T→	student_id	student_name	age	class	address	teacher_id
☐ Edit Copy Delete	1	Rana Vrujal	20	12C	Surat	NULL
☐ Edit Copy Delete	2	Amit Jangid	22	11A	Vadodara	NULL
☐ Edit Copy Delete	3	Riya Shah	14	9B	Rajkot	NULL
☐ Edit Copy Delete	4	Neha Joshi	13	8A	Rajkot	NULL
☐ Edit Copy Delete	5	Karan Mehta	15	10B	Bhavnagar	NULL
☐ Edit Copy Delete	6	Raj patel	9	9D	Jaipur	NULL
☐ Edit Copy Delete	7	Meet Patel	6	4A	Junagadh	NULL

4. Main SQL Commands and Sub-commands (DDL)

Theory Questions:

1. Define the SQL Data Definition Language (DDL).

DDL is a part of SQL used to create, change, or delete database structures like tables, databases, and indexes.

It helps you **define** how the database will look.

Examples of DDL commands (simple words)

- **CREATE** → to make a new table or database
- **ALTER** → to change a table
- **DROP** → to delete a table or database
- **TRUNCATE** → to remove all data from a table

2. Explain the CREATE command and its syntax.

The **CREATE** command in SQL is used to **make a new object** in the database. You can create things like **tables, databases, views, indexes**, etc.

Most commonly, it is used to **create a new table**.

Example: -

```
CREATE TABLE students (  
    student_id INT, student_name  
    VARCHAR(50), age INT  
);
```

3. What is the purpose of specifying data types and constraints during table creation?

1. Data Types

Data types are used to **decide what kind of data** can be stored in each column. Example: numbers, text, dates, decimals, etc.

- They help the database **store data correctly**
- They **save storage space**
- They **prevent wrong data** from being entered

2. Constraints

Introduction to DBMS

Constraints are rules applied to columns to **control what values are allowed**.

- They help keep the data **correct and safe**
- They prevent **duplicate, empty, or invalid** data
- They maintain **relationships** between tables

Example: -

- INT → only numbers
- VARCHAR → text
- NOT NULL → value cannot be empty
- UNIQUE → value cannot be repeated
- PRIMARY KEY → uniquely identifies each record
- FOREIGN KEY → links two tables

LAB EXERCISES:

Lab 1: Create a table courses with columns: course_id, course_name, and course_credits. Set the course_id as the primary key.

CREATE TABLE courses(course_id int PRIMARY KEY AUTO_INCREMENT, course_name varchar(100), course_credits int);

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0335 seconds.)

```
CREATE TABLE courses(course_id int PRIMARY KEY AUTO_INCREMENT, course_name varchar(100), course_credits int);
```

course_id	course_name	course_credits
-----------	-------------	----------------

Query results operations

Lab 2: Use the CREATE command to create a database university_db. CREATE DATABASE university_db;

✓ MySQL returned an empty result set (i.e. zero rows).

```
CREATE DATABASE university_db;
```

[Edit inline] [Edit] [Create PHP code]

5. ALTER Command

Theory Questions:

1. What is the use of the ALTER command in SQL?

The **ALTER** command is used to **change an existing table**. It helps you update the table structure **without deleting the table**.

What you can do with ALTER:

- Add a new column
- Delete a column
- Change a column's data type
- Rename a column or table
- Add or remove constraints (like PRIMARY KEY, FOREIGN KEY)

Example: -

```
ALTER TABLE students ADD email VARCHAR(50);
```

2. How can you add, modify, and drop columns from a table using ALTER?

1. Add a Column

Use **ADD** to insert a new column into an existing table.

```
ALTER TABLE table_name ADD column_name datatype;
```

Example:

```
ALTER TABLE students ADD  
email VARCHAR(100);
```

2. Modify (Change) a Column

Use **MODIFY** (MySQL) or **ALTER COLUMN** (SQL Server/PostgreSQL) to change a column's datatype or size.

MySQL Syntax:

```
ALTER TABLE table_name MODIFY column_name new_datatype;
```

Example:

```
ALTER TABLE students MODIFY email VARCHAR(150);
```

3. Drop (Delete) a Column

Use **DROP COLUMN** to remove a column from the table.

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Example:

ALTER TABLE students DROP COLUMN email;

- How to rename a column
- How to add or remove constraints using ALTER Just tell me!

LAB EXERCISES:

Lab 1: Modify the courses table by adding a column course_duration using the ALTER command.

ALTER TABLE courses ADD course_duration varchar(50);

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0818 seconds.)

```
ALTER TABLE courses ADD course_duration varchar(50);
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

course_id	course_name	course_credits	course_duration
-----------	-------------	----------------	-----------------

Lab 2: Drop the course_credits column from the courses table.

ALTER TABLE courses DROP COLUMN course_credits;

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

course_id	course_name	course_duration
-----------	-------------	-----------------

Query results operations

6. DROP Command

Theory Questions:

1. What is the function of the DROP command in SQL?

The **DROP** command is used to **permanently delete** a database object. It removes the object completely from the database.

- **You can DROP:**
 - A **table**
 - A **database**
 - A **view**
 - An **index**
 - A **column** (using ALTER TABLE ... DROP COLUMN)

- **Example:**

```
DROP TABLE students;
```

This deletes the entire table and all its data.

2. What are the implications of dropping a table from a database?

- 1. The table is permanently deleted**

You cannot recover it unless you have a backup.

- 2. All data inside the table is lost**

Every record stored in that table is removed forever.

- 3. All constraints linked to the table are removed**

Primary keys, foreign keys, and indexes related to that table are deleted.

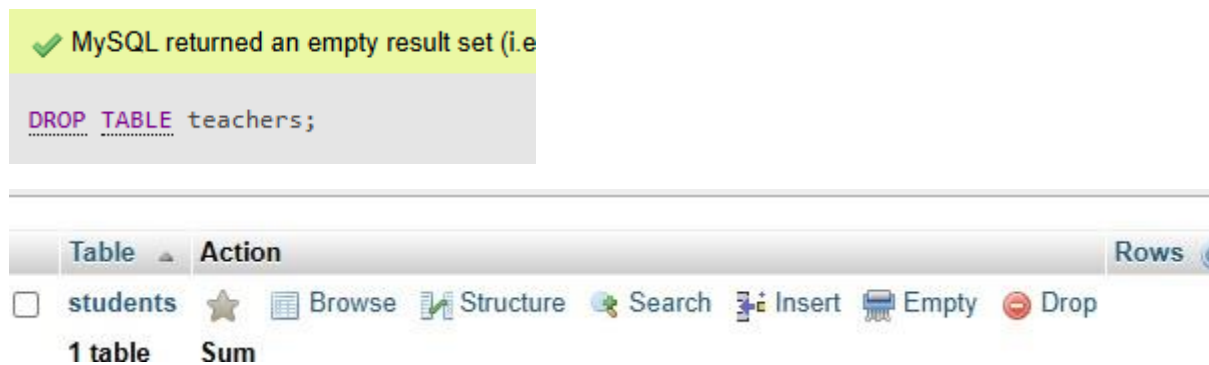
- 4. Other tables may be affected**

If any table depends on it through a foreign key, errors can occur unless the foreign key is removed first.

LAB EXERCISES:

Lab 1: Drop the teachers table from the school_db database.

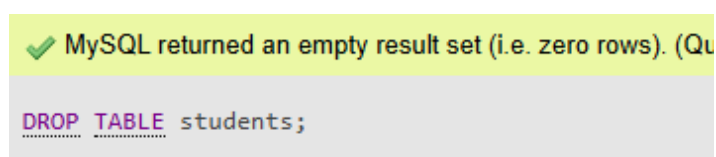
```
DROP TABLE teachers;
```



Lab 2: Drop the students table from the school_db database and verify that the table has been removed.

Drop the students table

```
DROP TABLE students;
```



Verify that the table has been removed

Use this command to list all tables:

```
SHOW TABLES;
```

7. Data Manipulation Language (DML)

Theory Questions:

1. Define the INSERT, UPDATE, and DELETE commands in SQL.

INSERT Command

The **INSERT** command is used to **add new rows** into a table. It stores new data in the database.

Example:

```
INSERT INTO table_name (columns) VALUES (values);
```

UPDATE Command

The **UPDATE** command is used to **change or modify existing data** in a table. It updates only the rows that match the condition.

Example:

```
UPDATE table_name SET column = value WHERE condition;
```

DELETE Command

The **DELETE** command is used to **remove rows** from a table. It deletes only the data you select with a condition.

Example:

```
DELETE FROM table_name WHERE condition;
```

2. What is the importance of the WHERE clause in UPDATE and DELETE operations?

The **WHERE** clause is very important in **UPDATE** and **DELETE** commands because it tells the database **which specific rows** should be updated or deleted.

Without the WHERE clause:

- **UPDATE** will change *all rows* in the table
- **DELETE** will remove *all rows* from the table

So, the WHERE clause helps to:

Introduction to DBMS

- Prevent accidental changes
- Protect the data
- Update or delete **only the required records**

LAB EXERCISES:

Lab 1: Insert three records into the courses table using the INSERT command.

```
INSERT INTO courses (course_name,course_duration)VALUES('web development','3 months'); INSERT  
INTO courses (course_name,course_duration)VALUES('maths','1 months');  
INSERT INTO courses (course_name,course_duration)VALUES('php','3 months');
```

			course_id	course_name	course_duration
☐	 Edit	 Copy	 Delete	1 web development	3 months
☐	 Edit	 Copy	 Delete	2 maths	1 months
☐	 Edit	 Copy	 Delete	3 php	3 months

Lab 2: Update the course duration of a specific course using the UPDATE command.

```
UPDATE courses SET course_duration='8 months' WHERE course_name='web development'; UPDATE  
courses SET course_duration='2 months' WHERE course_name='maths';  
UPDATE courses SET course_duration='5 months' WHERE course_name='php';
```

Show query box

✓ 1 row affected. (Query took 0.0212 seconds.)

```
UPDATE courses SET course_duration='8 months' WHERE course_name='web development';
```

[Edit inline] [Edit] [Create PHP code]

✓ 1 row affected. (Query took 0.0011 seconds.)

```
UPDATE courses SET course_duration='2 months' WHERE course_name='maths';
```

[Edit inline] [Edit] [Create PHP code]

✓ 1 row affected. (Query took 0.0011 seconds.)

```
UPDATE courses SET course_duration='5 months' WHERE course_name='php';
```

[Edit inline] [Edit] [Create PHP code]

				course_id	course_name	course_duration
☐				1	web development	8 months
☐				2	maths	2 months
☐				3	php	5 months

Lab 3: Delete a course with a specific course_id from the courses table using the DELETE command.

```
DELETE FROM courses WHERE course_id=2;
```

				course_id	course_name	course_duration
☐				1	web development	8 months
☐				3	php	5 months

8. Data Query Language (DQL)

Theory Questions:

1. What is the SELECT statement, and how is it used to query data?

The **SELECT statement** is used to **get data** from a table in a database. It helps you choose which information you want to see.

Example:

```
SELECT * FROM courses;
```

This shows **all the data** from the courses table.

In Easy Words:

- **SELECT = show data**
- It helps you **look inside** a table.
- You can choose **specific columns** or **specific rows**.

2. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

The **WHERE** clause is used to **filter data**.
It shows only the rows that **match a condition**.

Easy Words:

- **WHERE = filter the data**
- Shows only the records you want.

Example:

```
SELECT * FROM courses  
WHERE course_id = 2;
```

☐ Use of the ORDER BY Clause

The **ORDER BY** clause is used to **sort data**.
You can sort data in **ascending (ASC)** or **descending (DESC)** order.

Easy Words:

- **ORDER BY = sort the data**
- Helps arrange results from small to big or A to Z.

Example:

```
SELECT * FROM courses ORDER BY course_name ASC;
```

LAB EXERCISES:

Lab 1: Retrieve all courses from the courses table using the SELECT statement.

```
SELECT *  
FROM courses;
```

		course_id	course_name	course_duration
☐	Edit	Copy	Delete	
		1	web development	8 months
☐	Edit	Copy	Delete	
		4	python	7 months
☐	Edit	Copy	Delete	
		3	php	5 months
☐	Edit	Copy	Delete	
		5	mysql	1 months
☐	Edit	Copy	Delete	
		6	html	2 months

Lab 2: Sort the courses based on course_duration in descending order using ORDER BY. SELECT *
FROM courses ORDER BY course_duration DESC;

		course_id	course_name	course_duration
☐	Edit	Copy	Delete	
		1	web development	8 months
☐	Edit	Copy	Delete	
		4	python	7 months
☐	Edit	Copy	Delete	
		3	php	5 months
☐	Edit	Copy	Delete	
		6	html	2 months
☐	Edit	Copy	Delete	
		5	mysql	1 months

Lab 3: Limit the results of the SELECT query to show only the top two courses using LIMIT. SELECT *
FROM courses LIMIT 2;

		course_id	course_name	course_duration
☐	Edit	Copy	Delete	
		1	web development	8 months
☐	Edit	Copy	Delete	
		4	python	7 months

9. Data Control Language (DCL)

Theory Questions:

1. What is the purpose of GRANT and REVOKE in SQL?

Purpose of GRANT in SQL

GRANT is used to **give permissions** to a user.

It allows a user to perform actions like SELECT, INSERT, UPDATE, DELETE, etc.

Easy Words:

- **GRANT** = give permission

Purpose of REVOKE in SQL

REVOKE is used to **take back permissions** from a user. It removes the rights that were earlier given.

Easy Words:

- **REVOKE = remove permission**

2. How do you manage privileges using these commands?

You manage privileges by **giving** and **removing** permissions for users in the database.

Easy Words:

- Use **GRANT** to **allow** a user to do something.
- Use **REVOKE** to **stop** a user from doing something.

Example:

Give permission (GRANT):

```
GRANT SELECT, INSERT ON courses TO user1;
```

This allows **user1** to select and insert data in the courses table.

Remove permission (REVOKE):

```
REVOKE INSERT ON courses FROM user1;
```

This removes the INSERT right from **user1**.

LAB EXERCISES:

Lab 1: Create two new users user1 and user2 and grant user1 permission to SELECT from the courses table. CREATE USER 'user1'@'localhost' IDENTIFIED BY 'password1';

```
CREATE USER 'user2'@'localhost' IDENTIFIED BY 'password2'; GRANT  
SELECT ON university_db.courses TO 'user1'@'localhost';
```

Lab 2: Revoke the INSERT permission from user1 and give it to user2.

```
REVOKE INSERT ON school_db.courses FROM 'user1'@'localhost';
```

Extra options

Grants for user1@localhost

```
GRANT USAGE ON *.* TO `user1`@`localhost`
```

```
GRANT SELECT ON `university_db`.`courses` TO `user...
```

Grants for user2@localhost

```
GRANT USAGE ON *.* TO `user2`@`localhost`
```

```
GRANT INSERT ON `university_db`.`courses` TO `user...
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0234 seconds.)

```
GRANT INSERT ON university_db.courses TO 'user2'@'localhost';
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0050 seconds.)

```
GRANT INSERT ON university_db.courses TO 'user1'@'localhost';
```

[Edit inline](#)] [\[Edit \]](#) [\[Create PHP code \]](#)

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0040 seconds.)

```
REVOKE INSERT ON university_db.courses FROM 'user1'@'localhost';
```

[Edit inline](#)] [\[Edit \]](#) [\[Create PHP code \]](#)

10. Transaction Control Language (TCL)

Theory Questions:

1. What is the purpose of the COMMIT and ROLLBACK commands in SQL?

COMMIT

- **Definition:** COMMIT is used to **save all the changes** made in the current transaction permanently to the database.
- **Purpose:** Ensures that your changes (INSERT, UPDATE, DELETE) are **final and cannot be undone**.
- **Example:**

```
START TRANSACTION;
```

```
INSERT INTO courses (course_id, course_name) VALUES (1, 'Mathematics'); INSERT  
INTO courses (course_id, course_name) VALUES (2, 'Physics');
```

```
COMMIT; -- Changes are saved permanently
```

- **Key point:** Once you commit, the changes are permanent.

ROLLBACK

- **Definition:** ROLLBACK is used to **undo changes** made in the current transaction.

- **Purpose:** It **reverts the database to its previous state**, before the transaction started.
- **Example:**

START TRANSACTION;

INSERT INTO courses (course_id, course_name) VALUES (3, 'Chemistry');

ROLLBACK; -- The insertion is undone

- **Key point:** Useful when something goes wrong, like an error in data or logic.

Summary Table

Command	Purpose	Effect on Database
COMMIT	Save all changes permanently	Changes cannot be undone
ROLLBACK	Undo all changes in the current transaction	Database returns to previous state

In short:

- **COMMIT** → **Save changes permanently**
- **ROLLBACK** → **Cancel changes and restore old data**

2. Explain how transactions are managed in SQL databases.

What is a Transaction?

- A **transaction** is a **logical unit of work** in a database.
- It consists of one or more SQL statements that must **either all succeed or all fail**.
- Example: Transferring money from one account to another:
 1. Deduct amount from Account A
 2. Add amount to Account B→ Both steps must succeed, or neither should happen.

ACID Properties

Transactions are managed using **ACID principles** to ensure reliability:

Property	Meaning
A – Atomicity	All operations in a transaction succeed or fail as a single unit.
C – Consistency	Database moves from one valid state to another after a transaction.
I – Isolation	Transactions are executed independently without interfering with each other.
D – Durability	Once a transaction is committed, the changes are permanent even if the system crashes.

Transaction Control Commands

SQL provides commands to manage transactions:

Command	Purpose
START TRANSACTION / BEGIN	Begin a new transaction
COMMIT	Save all changes permanently
ROLLBACK	Undo changes made in the current transaction
SAVEPOINT	Set a point in a transaction to roll back to partially

How Transactions Are Managed

1. **Start Transaction**
2. `START TRANSACTION;`
3. **Execute SQL statements**
4. `INSERT INTO accounts VALUES (1, 'A', 5000);`
5. `UPDATE accounts SET balance = balance - 500 WHERE id = 1;`
6. `UPDATE accounts SET balance = balance + 500 WHERE id = 2;`
7. **Check for errors**
 - If everything is correct → `COMMIT`
 - If an error occurs → `ROLLBACK`
8. **Optional SAVEPOINT**
 - Allows partial rollback within a transaction:
9. `SAVEPOINT sp1;`
10. `-- some operations`
11. `ROLLBACK TO sp1;`

Example

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 1000 WHERE id = 1; UPDATE  
accounts SET balance = balance + 1000 WHERE id = 2;
```

```
-- Suppose an error occurs here  
ROLLBACK; -- Undo both updates
```

```
-- If no error  
COMMIT; -- Save changes permanently
```

Summary

- Transactions ensure **data integrity**.
- SQL provides commands (**START TRANSACTION, COMMIT, ROLLBACK**) to **control changes safely**.
- ACID properties guarantee **reliability, consistency, and durability**.

LAB EXERCISES:

Lab 1: Insert a few rows into the courses table and use `COMMIT` to save the changes. `START`

```
TRANSACTION;
```

```
INSERT INTO courses (course_name,course_duration) VALUES ('java','7 months');
```


Introduction to DBMS

```
INSERT INTO courses (course_name,course_duration) VALUES ('mern','10 months'); INSERT
```

```
INTO courses (course_name,course_duration) VALUES ('css',' 2 months'); COMMIT;
```

			course_id	course_name	course_duration
☐	 Edit	 Copy	 Delete	1	web development 8 months
☐	 Edit	 Copy	 Delete	4	python 7 months
☐	 Edit	 Copy	 Delete	3	php 5 months
☐	 Edit	 Copy	 Delete	5	mysql 1 months
☐	 Edit	 Copy	 Delete	6	html 2 months
☐	 Edit	 Copy	 Delete	7	java 7 moths
☐	 Edit	 Copy	 Delete	8	mern 10 moths
☐	 Edit	 Copy	 Delete	9	css 2 moths

Lab 2: Insert additional rows, then use ROLLBACK to undo the last insert operation. INSERT INTO

```
courses (course_name,course_duration) VALUES ('Biology', '5 months');
```

```
INSERT INTO courses (course_name,course_duration) VALUES ('Computer Science', '8 months'); ROLLBACK;
```

				course_id	course_name	course_duration
☐	 Edit	 Copy	 Delete	1	web development	8 months
☐	 Edit	 Copy	 Delete	4	python	7 months
☐	 Edit	 Copy	 Delete	3	php	5 months
☐	 Edit	 Copy	 Delete	5	mysql	1 months
☐	 Edit	 Copy	 Delete	6	html	2 months
☐	 Edit	 Copy	 Delete	7	java	7 moths
☐	 Edit	 Copy	 Delete	8	mern	10 moths
☐	 Edit	 Copy	 Delete	9	css	2 moths
☐	 Edit	 Copy	 Delete	10	Biology	5 months
☐	 Edit	 Copy	 Delete	11	Computer Science	8 months

Lab 3: Create a SAVEPOINT before updating the courses table, and use it to roll back specific changes.

		course_id	course_name	course_duration
☐	Edit Copy Delete	1	web development	8 months
☐	Edit Copy Delete	4	python	7 months
☐	Edit Copy Delete	3	php	5 months
☐	Edit Copy Delete	5	mysql	3 month
☐	Edit Copy Delete	6	html	2 months
☐	Edit Copy Delete	7	java	7 moths
☐	Edit Copy Delete	8	mern	10 moths
☐	Edit Copy Delete	9	css	2 moths
☐	Edit Copy Delete	10	Biology	5 months
☐	Edit Copy Delete	11	Computer Science	8 months

11. SQL Joins

Theory Questions:

1. Explain the concept of JOIN in SQL. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

Types of JOIN and Their Differences

1 INNER JOIN

- Returns **only the matching rows** from both tables.
- Rows that do not match are **excluded**.

2 LEFT JOIN (LEFT OUTER JOIN)

- Returns **all rows from the left table**, and the **matching rows** from the right table.
- If no match exists, values from the right table are shown as **NULL**.

3 RIGHT JOIN (RIGHT OUTER JOIN)

- Returns **all rows from the right table**, and the **matching rows** from the left table.
- If no match exists, values from the left table are shown as **NULL**.

4 FULL OUTER JOIN

- Returns **all rows from both tables**.
- Matching rows are combined.
- Non-matching rows from both tables are shown with **NULL** values.

Difference in One Simple Table

JOIN Type	Shows Matching Rows	Shows Unmatched Left	Shows Unmatched Right
INNER JOIN	✓	✗	✗
LEFT JOIN	✓	✓	✗
RIGHT JOIN	✓	✗	✓
FULL OUTER JOIN	✓	✓	✓

2. How are joins used to combine data from multiple tables?

Joins are used to combine data from multiple tables by matching rows based on a **common column** (usually a primary key–foreign key relationship).

SQL reads both tables, finds the related values, and returns a single result set containing data from all the joined tables.

How it works

1. Each table stores different information (e.g., students in one table and classes in another).
2. A JOIN compares the related columns (like class_id).
3. It combines the matching rows into one output using the join condition:

```
SELECT columns  
FROM table1 JOIN  
table2  
ON table1.common_column = table2.common_column;
```

Why joins are used

- To avoid storing duplicate data
- To connect related information
- To retrieve complete results from normalized databases

LAB EXERCISES:

Lab 1: Create two tables: departments and employees. Perform an INNER JOIN to display employees along with their respective departments.

Department table: -

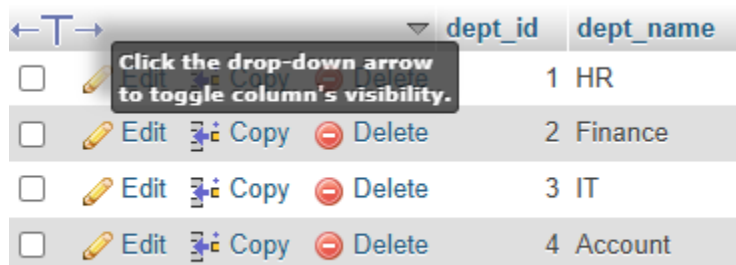
Table create: -

```
CREATE TABLE departments (dept_id int PRIMARY KEY AUTO_INCREMENT, dept_name varchar(50));  
Data insert: -
```

```
INSERT INTO departments (dept_name) VALUES ('HR');
```

Introduction to DBMS

```
INSERT INTO departments (dept_name) VALUES ('Finance');  
INSERT INTO departments (dept_name) VALUES ('IT'); INSERT  
INTO departments (dept_name) VALUES ('Account');
```



Click the drop-down arrow to toggle column's visibility.

		dept_id	dept_name
☐		1	HR
☐	Edit Copy Delete	2	Finance
☐	Edit Copy Delete	3	IT
☐	Edit Copy Delete	4	Account

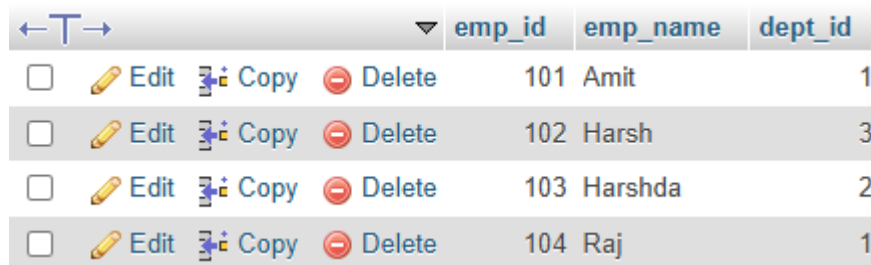
Employees table: -

Table: -

```
CREATE TABLE employees(emp_id int PRIMARY KEY AUTO_INCREMENT, emp_name varchar(50), dept_id int,  
FOREIGN KEY(dept_id) REFERENCES departments(dept_id));
```

Data insert: -

```
INSERT INTO employees (emp_name,dept_id) VALUES ('Amit',1); INSERT  
INTO employees (emp_name,dept_id) VALUES ('Harsh',3); INSERT INTO  
employees (emp_name,dept_id) VALUES ('Harshda',2); INSERT INTO  
employees (emp_name,dept_id) VALUES ('Raj',1);
```



		emp_id	emp_name	dept_id
☐	Edit Copy Delete	101	Amit	1
☐	Edit Copy Delete	102	Harsh	3
☐	Edit Copy Delete	103	Harshda	2
☐	Edit Copy Delete	104	Raj	1

Run SQL query/queries on database join:

```
1 SELECT employees.emp_id,employees.emp_name,departments.dept_name FROM employees INNER JOIN departments ON employees.dept_id =  
departments.dept_id
```

```
SELECT employees.emp_id,employees.emp_name,departments.dept_name FROM employees INNER JOIN departments ON  
employees.dept_id = departments.dept_id
```

emp_id	emp_name	dept_name
101	Amit	HR
102	Harsh	IT
103	Harshda	Finance
104	Raj	HR

Lab 2: Use a LEFT JOIN to show all departments, even those without employees.

```
SELECT departments.dept_id, departments.dept_name, employees.emp_name FROM departments LEFT JOIN employees ON departments.dept_id = employees.dept_id;
```

dept_id	dept_name	emp_name
1	HR	Raj
1	HR	Amit
2	Finance	Harshda
3	IT	Harsh
4	Account	NULL

12. SQL Group By

Theory Questions:

1. What is the GROUP BY clause in SQL? How is it used with aggregate functions?

The **GROUP BY** clause in SQL is used to group rows that have the **same values** in one or more columns. It allows you to organize data into groups so that you can apply **aggregate functions** to each group separately.

Aggregate functions include:

- **COUNT()** – counts rows
- **SUM()** – adds values
- **AVG()** – finds average
- **MAX()** – highest value
- **MIN()** – lowest value

When GROUP BY is used with these functions, SQL returns **one result per group** instead of one result per row.

✓ Example

```
SELECT department, COUNT(emp_id) AS total_employees FROM employees GROUP BY department;
```

Explanation

- The department column is used to create groups.
- COUNT(emp_id) calculates the number of employees **in each department**.
- The output displays each department **once**, along with its total employees.

✓ Perfect Summary

- **GROUP BY** groups similar rows.
- Aggregate functions calculate values for **each group**.
- Commonly used for reports like totals, averages, and counts.

2. Explain the difference between GROUP BY and ORDER BY.

GROUP BY

- Used to **group rows** that have the **same values** in one or more columns.
- Mostly used with **aggregate functions** like COUNT(), SUM(), AVG(), MAX(), MIN().
- Output shows **one row per group**.

Purpose:

To **summarize** or **aggregate** data.

Example:

Group employees by department.

```
SELECT department, COUNT(*) FROM  
employees  
GROUP BY department;
```

ORDER BY

- Used to **sort the result** in ascending (ASC) or descending (DESC) order.
- Does **not** perform grouping or aggregation.
- Works on individual rows or on grouped results.

Purpose:

To **arrange** data in a specific order.

Example:

Sort employees by name.

```
SELECT * FROM employees ORDER  
BY emp_name ASC;
```

✓ Key Differences

Feature	GROUP BY	ORDER BY
Function	Groups rows	Sorts rows
Used with	Aggregate functions	Any column or expression
Output	One row per group	Same rows, just sorted

Feature	GROUP BY	ORDER BY
Order of Use	Comes before ORDER BY in SQL	Comes after GROUP BY

✓ Perfect Summary

- **GROUP BY = Grouping + Aggregation**
- **ORDER BY = Sorting the output**

LAB EXERCISES:

Lab 1: Group employees by department and count the number of employees in each department using GROUP BY.

SELECT dept_id, COUNT(emp_id) AS total_employees FROM employees GROUP BY dept_id;

dept_id	total_employees
1	2
2	1
3	1

Lab 2: Use the AVG aggregate function to find the average salary of employees in each department

ALTER TABLE employees ADD salary INT;

UPDATE employees SET salary = 30000 WHERE emp_id = 101;

UPDATE employees SET salary = 45000 WHERE emp_id = 102;

UPDATE employees SET salary = 50000 WHERE emp_id = 103;

UPDATE employees SET salary = 32000 WHERE emp_id = 104;

	emp_id	emp_name	dept_id	salary
☐ Edit Copy Delete	101	Amit	1	30000
☐ Edit Copy Delete	102	Harsh	3	45000
☐ Edit Copy Delete	103	Harshda	2	50000
☐ Edit Copy Delete	104	Raj	1	32000

SELECT dept_id, AVG(salary) AS average_salary FROM employees GROUP BY dept_id;

	dept_id	average_salary
☐ Edit Copy Delete	1	31000.0000
☐ Edit Copy Delete	2	50000.0000
☐ Edit Copy Delete	3	45000.0000

13. SQL Stored Procedure

Theory Questions:

1. What is a stored procedure in SQL, and how does it differ from a standard SQL query?

Stored Procedure (Definition)

A **stored procedure** is a pre-written, reusable block of SQL code that is stored in the database and can be executed whenever needed.

It may contain multiple SQL statements such as SELECT, INSERT, UPDATE, DELETE, along with logic like conditions and loops.

✓ How it differs from a standard SQL query

Feature	Stored Procedure	Standard SQL Query
Definition	Predefined program stored in the database	A single SQL command written and executed manually
Execution	Run using a call (e.g., CALL procedure_name;)	Executed directly in the SQL editor
Reusability	Highly reusable; written once, used many times	Must be written each time you want to run it
Performance	Faster — compiled and cached by the database	Slower — executed and parsed each time
Complexity	Can include multiple statements, loops, conditions	Usually one simple statement
Security	Can restrict direct table access and allow controlled execution	Gives direct access to tables

✓ Perfect Summary

- A **stored procedure** is a reusable program saved in the database.
- A **standard SQL query** is a one-time command you execute manually.
- Stored procedures are faster, more secure, and used for automation.

2. Explain the advantages of using stored procedures.

Stored procedures offer several benefits that make database operations faster, safer, and more organized.

☐ Improved Performance

- Stored procedures are **precompiled** and stored in the database.
- They run faster than writing SQL queries again and again.

☐ Reusability

- Write once, use many times.

- The same procedure can be called multiple times by different programs or users.

3 Better Security

- Users can be given permission to **execute** a procedure **without accessing tables directly**.
- Helps protect sensitive data.

4 Reduced Network Traffic

- Instead of sending multiple SQL queries over the network, the client only sends:
CALL procedure_name;
- This reduces data transfer and speeds up execution.

5 Maintains Consistency

- Business logic is stored in one place.
- Ensures that all programs follow the same rules and calculations.

6 Easier Maintenance

- If logic changes, update the stored procedure once.
- All applications automatically use the updated logic.

LAB EXERCISES:

Lab 1: Write a stored procedure to retrieve all employees from the employees table based on department.

DELIMITER \$\$

```
CREATE PROCEDURE GetEmployeesByDept(IN dept INT) BEGIN
```

```
    SELECT emp_id, emp_name, dept_id FROM
```

```
    employees
```

```
    WHERE dept_id = dept;
```

```
END$$
```

DELIMITER ;

```
CALL GetEmployeesByDept(1);
```

emp_id	emp_name	dept_id
101	Amit	1
104	Raj	1

Lab 2: Write a stored procedure that accepts course_id as input and returns the course details

DELIMITER \$\$

```
CREATE PROCEDURE GetEmployeesByDept(IN dept INT)
```

```
BEGIN
```

```
    SELECT emp_id, emp_name, dept_id FROM
```

```
    employees
```

```
    WHERE dept_id = dept;
```

```
END$$
```

DELIMITER ;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0250 seconds.)

```
CREATE PROCEDURE GetCourseDetails(IN course_id_input INT) BEGIN SELECT * FROM courses WHERE course_id = course_id_input; END;
```

```
CALL GetCourseDetails(1);
```

course_id	course_name	course_duration
1	web development	8 months

14. SQL View

Theory Questions:

1. What is a view in SQL, and how is it different from a table?

A **view** is a **virtual table** in SQL that is created from a SELECT query. It does **not store data physically**; it only shows data stored in other tables.

✓ How is a View different from a Table?

View

Virtual table (logical)

Table

Physical table (actual data stored)

View	Table
Stores no data	Stores real data
Created using CREATE VIEW	Created using CREATE TABLE
Automatically updates when underlying table changes	Data changes only when inserted/updated manually

2. Explain the advantages of using views in SQL databases.

- **Security**

- Views allow you to hide sensitive columns and show only required data to specific users.

- **Simplicity**

- Complex queries can be saved as a view, making it easy to reuse with a simple SELECT.

- **Data Consistency**

- Since views pull data from base tables, the displayed data is always up-to-date.

- **Logical Independence**

- You can change the underlying table structure without affecting user queries using the view.

- **Reusability**

- Once created, views can be used multiple times like a table, saving time.

- **Improves Query Readability**

- Reduces complexity and makes long SQL queries easier to understand.

LAB EXERCISES:

Lab 1: Create a view to show all employees along with their department names.

```
CREATE VIEW employee_department_view AS SELECT  
e.emp_id,  
e.emp_name,  
d.dept_name FROM  
employees e JOIN  
departments d
```

Introduction to DBMS

ON e.dept_id = d.dept_id;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0439 seconds.)

```
CREATE VIEW employee_department_view AS SELECT e.emp_id, e.emp_name, d.dept_name FROM employees e JOIN departments d ON e.dept_id = d.dept_id;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

SELECT * FROM employee_department_view;

				emp_id	emp_name	dept_name
☐		Edit		Copy		Delete
				101	Amit	HR
☐		Edit		Copy		Delete
				102	Harsh	IT
☐		Edit		Copy		Delete
				103	Harshda	Finance
☐		Edit		Copy		Delete
				104	Raj	HR

Lab 2: Modify the view to exclude employees whose salaries are below \$50,000.

First: -

DROP VIEW IF EXISTS employee_department_view;

Second: -

CREATE VIEW employee_department_view AS SELECT

e.emp_id,

e.emp_name,

d.dept_name FROM

employees e JOIN

departments d

ON e.dept_id = d.dept_id WHERE

e.salary >= 50000;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0123 seconds.)

```
CREATE VIEW employee_department_view AS SELECT e.emp_id, e.emp_name, d.dept_name FROM employees e JOIN departments d ON e.dept_id = d.dept_id WHERE e.salary >= 50000;
```

SELECT * FROM employee_department_view;

				emp_id	emp_name	dept_name
☐		Edit		Copy		Delete
				103	Harshda	Finance

15. SQL Triggers

Theory Questions:

1. What is a trigger in SQL? Describe its types and when they are used.

1. What is a Trigger in SQL?

A trigger is an automatic procedure in SQL that runs when a specific event happens on a table, such as INSERT, UPDATE, or DELETE.

Triggers help automate tasks, maintain data integrity, and perform auditing.

Types of Triggers and When They Are Used

1. BEFORE Trigger

Runs before an INSERT, UPDATE, or DELETE occurs.

Used for:

- Validating data
- Modifying values before saving
- Preventing invalid actions

2. AFTER Trigger

Runs after an INSERT, UPDATE, or DELETE completes.

Used for:

- Storing logs
- Auditing changes
- Sending notifications or updates

3. INSTEAD OF Trigger

Used mainly with views and runs instead of the actual operation.

Used for:

- Handling changes on complex views
- Custom customized logic for insert/update/delete on views

2. Explain the difference between INSERT, UPDATE, and DELETE triggers.

Trigger Type	When It Executes	What It Is Used For	Example Use Case
INSERT Trigger	When a new row is inserted	Validate new data, set default values, create log records	Add a log entry whenever a new employee is added
UPDATE Trigger	When an existing row is updated	Track changes, store old & new values, validate updates	Save old salary before updating to new salary

Trigger Type	When It Executes	What It Is Used For	Example Use Case
DELETE Trigger	When a row is deleted	Prevent accidental deletes, create delete logs, backup deleted data	Move deleted records to a backup table

LAB EXERCISES:

Lab 1: Create a trigger to automatically log changes to the employees table when a new employee is added.

DELIMITER \$\$

CREATE TRIGGER log_new_employee AFTER

INSERT ON employees

FOR EACH ROW

BEGIN

INSERT INTO employee_log (emp_id, action, action_time) VALUES

(NEW.emp_id, 'New Employee Added', NOW());

END\$\$

DELIMITER ;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0415 seconds.)

```
CREATE TRIGGER log_new_employee AFTER INSERT ON employees FOR EACH ROW BEGIN INSERT INTO employee_log (emp_id, action, action_time) VALUES (NEW.emp_id, 'New Employee Added', NOW()); END;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

SHOW TRIGGERS;

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_client	collation_connection	Database Collation
log_new_employee	INSERT	employees	BEGIN INSERT INTO employee_log (emp_id, actio...	AFTER	2025-11-30 19:36:55.18		root@localhost	utf8mb4	utf8mb4_unicode_ci	utf8mb4_0900_ai_ci

SHOW TRIGGERS LIKE 'employees';

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_client	collation_connection	Database Collation
log_new_employee	INSERT	employees	BEGIN INSERT INTO employee_log (emp_id, actio...	AFTER	2025-11-30 19:36:55.18		root@localhost	utf8mb4	utf8mb4_unicode_ci	utf8mb4_0900_ai_ci

SHOW CREATE TRIGGER log_new_employee;

Lab 2: Create a trigger to update the last_modified timestamp whenever an employee record is updated.

Introduction to DBMS

First to add the new column: -

```
ALTER TABLE employees ADD COLUMN last_modified DATETIME;
```

```
DELIMITER $$
```

```
CREATE TRIGGER update_last_modified
```

```
BEFORE UPDATE ON employees
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    SET NEW.last_modified = NOW();
```

```
END$$
```

```
DELIMITER ;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0308 seconds.)

```
CREATE TRIGGER update_last_modified BEFORE UPDATE ON employees FOR EACH ROW BEGIN SET NEW.last_modified = NOW(); END;
```

```
SHOW TRIGGERS LIKE 'employees';
```

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_client	collation_connection	Database Collation
log_new_employee	INSERT	employees	BEGIN INSERT INTO employee_log (emp_id, actio...	AFTER	2025-11-30 19:36:55.18		root@localhost	utf8mb4	utf8mb4_unicode_ci	utf8mb4_0900_ai
update_last_modified	UPDATE	employees	BEGIN SET NEW.last_modified = NOW(); END	BEFORE	2025-11-30 19:49:56.23		root@localhost	utf8mb4	utf8mb4_unicode_ci	utf8mb4_0900_ai

```
UPDATE employees SET salary = 200000 WHERE emp_id = 101;
```

```
SELECT emp_id, emp_name, salary, last_modified FROM employees WHERE emp_id = 101;
```

	emp_id	emp_name	salary	last_modified
☐ Edit <input type="plus"/> Copy <input type="minus"/> Delete	101	Amit	200000	2025-11-30 20:02:52

16. Introduction to PL/SQL

Theory Questions:

1. What is PL/SQL, and how does it extend SQL's capabilities?

What is PL/SQL?

PL/SQL (Procedural Language/SQL) is Oracle's **procedural extension of SQL**.

It allows you to write **blocks of code** with variables, loops, and control structures, which standard SQL alone cannot do.

How PL/SQL Extends SQL's Capabilities

1. **Procedural Programming** – Supports IF...ELSE, loops (FOR, WHILE), and exception handling.
2. **Variables and Constants** – Store values temporarily for computations.
3. **Modular Programming** – Create procedures, functions, and packages for reusable code.
4. **Error Handling** – Use EXCEPTION blocks to manage runtime errors.
5. **Automation** – Combine multiple SQL statements in a single block for batch operations.

2. List and explain the benefits of using PL/SQL.

Benefits of Using PL/SQL

1. **Combines SQL and Procedural Features**
 - Allows using SQL for database operations and procedural constructs like loops, conditions, and variables.
2. **Improves Performance**
 - Multiple SQL statements can be executed in a single PL/SQL block, reducing network traffic.
3. **Reusability and Modularity**
 - Supports **procedures, functions, packages, and triggers** for reusable code.
4. **Error Handling**
 - Provides EXCEPTION blocks to handle runtime errors gracefully.
5. **Enhanced Security**
 - Code can be stored in the database as procedures/functions, hiding business logic from users.
6. **Easier Maintenance**
 - Modular blocks make updates and debugging simpler.
7. **Transaction Control**
 - Supports COMMIT, ROLLBACK, and SAVEPOINT to manage transactions efficiently.

LAB EXERCISES:

Lab 1: Write a PL/SQL block to print the total number of employees from the employees table.

DELIMITER \$\$

```
CREATE PROCEDURE get_total_employees() BEGIN
```

```
    DECLARE total_employees INT;
```


Introduction to DBMS

```
SELECT COUNT(*) INTO total_employees FROM  
employees;
```

```
SELECT CONCAT('Total number of employees: ', total_employees) AS result; END$$
```

DELIMITER ;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0300 seconds.)

```
CREATE PROCEDURE get_total_employees() BEGIN DECLARE total_employees INT; SELECT COUNT(*) INTO total_employees FROM employees; SELECT CONCAT('Total  
number of employees: ', total_employees) AS result; END;
```

```
CALL get_total_employees();
```

result

Total number of employees: 6

Lab 2: Create a PL/SQL block that calculates the total sales from an orders table.

```
CREATE TABLE orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT, order_amount  
    DECIMAL(10,2), order_date  
    DATE  
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0398 seconds.)

```
CREATE TABLE orders ( order_id INT AUTO_INCREMENT PRIMARY KEY, customer_id INT, order_amount DECIMAL(10,2), order_date DATE );
```

```
INSERT INTO orders (customer_id, order_amount, order_date)
```

```
VALUES
```

```
(1, 250.50, '2025-11-01'),
```

```
(2, 450.00, '2025-11-05'),
```

```
(1, 300.75, '2025-11-10');
```

				order_id	customer_id	order_amount	order_date
☐				1	1	250.50	2025-11-01
☐				2	2	450.00	2025-11-05
☐				3	1	300.75	2025-11-10

DELIMITER \$\$

```
CREATE PROCEDURE get_total_sales()
```

```
BEGIN
```

```
    DECLARE total_sales DECIMAL(10,2);
```

```
    SELECT SUM(order_amount) INTO total_sales
```

```
    FROM orders;
```

```
    SELECT CONCAT('Total sales: ', total_sales) AS result; END$$
```

DELIMITER ;

-- Call the procedure

```
CALL get_total_sales();
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0256 seconds.)

```
CREATE PROCEDURE get_total_sales() BEGIN DECLARE total_sales DECIMAL(10,2); SELECT SUM(order_amount) INTO total_sales FROM orders; SELECT  
CONCAT('Total sales: ', total_sales) AS result; END;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✓ Showing rows 0 - 0 (1 total, Query took 0.0037 seconds.)

```
-- Call the procedure CALL get_total_sales();
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

☐ Show all | Number of rows: 25 ▼ Filter rows:

Extra options

result

Total sales: 1001.25

17. PL/SQL Control Structures

Theory Questions:

1. What are control structures in PL/SQL? Explain the IF-THEN and LOOP control structures.

Control Structures in PL/SQL

Control structures are **programming constructs** in PL/SQL that allow you to **control the flow of execution** in a program. They help you make decisions, repeat statements, or handle exceptions.

1. IF-THEN Control Structure

Used to execute statements **conditionally** based on a boolean expression.

Syntax:

```
IF condition THEN
    -- statements to execute if condition is TRUE ELSIF another_condition
THEN
    -- statements if another_condition is TRUE ELSE
    -- statements if all conditions are FALSE END IF;
```

Example:

```
DECLARE
    salary NUMBER := 60000; BEGIN
    IF salary > 50000 THEN
        DBMS_OUTPUT.PUT_LINE('High salary');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Low salary');
    END IF;
END;
/
```

2. LOOP Control Structure

Used to **repeat a block of statements** multiple times until a condition is met.

Types of loops in PL/SQL:

1. Simple LOOP– indefinite loop, must use EXIT to stop.
2. WHILE loop – repeats while a condition is true.
3. FOR loop – repeats a fixed number of times.

Simple LOOP Example:

```
DECLARE
    i NUMBER := 1; BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE('Count: ' || i); i := i + 1;
        EXIT WHEN i > 5; END
    LOOP;
END;
/
```

Output:

Count: 1

Count: 2

Count: 3

Count: 4

Count: 5

2. How do control structures in PL/SQL help in writing complex queries?

1. **Conditional Execution**

- Using IF-THEN-ELSE, you can run different SQL statements depending on specific conditions.
- Example: Apply different discounts based on order amount.

2. **Repetition of Tasks**

- LOOP, WHILE, and FOR allow executing SQL statements multiple times automatically.
- Example: Process all employees' salaries in a loop without writing multiple queries.

3. **Dynamic Decision Making**

- Control structures enable decision-making within a block, making queries adaptable to different situations.
- Example: Update only employees meeting certain criteria dynamically.

4. **Modular and Readable Code**

- Instead of writing long, repeated SQL statements, control structures make your code organized and easier to maintain.

5. **Error Handling Integration**

- Can be combined with EXCEPTION blocks to handle errors while executing multiple or conditional SQL operations.

LAB EXERCISES:

Lab 1: Write a PL/SQL block using an IF-THEN condition to check the department of an employee.

DELIMITER \$\$

```
CREATE PROCEDURE check_employee_department(IN empId INT) BEGIN
```

```
    DECLARE emp_dept VARCHAR(50);
```

```
    SELECT d.dept_name INTO emp_dept FROM
```

```
    employees e
```

```
    JOIN departments d ON e.dept_id = d.dept_id WHERE
```

```
    e.emp_id = empId;
```

```
    IF emp_dept = 'Sales' THEN
```

```
        SELECT 'Employee belongs to Sales department' AS result; ELSE
```

```
        SELECT 'Employee belongs to another department' AS result;
```

```
END IF;
```

```
END$$
```

```
DELIMITER ;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0262 seconds.)

```
CREATE PROCEDURE check_employee_department(IN empId INT) BEGIN DECLARE emp_dept VARCHAR(50); SELECT d.dept_name INTO emp_dept FROM employees e JOIN departments d ON e.dept_id = d.dept_id WHERE e.emp_id = empId; IF emp_dept = 'Sales' THEN SELECT 'Employee belongs to Sales department' AS result; ELSE SELECT 'Employee belongs to another department' AS result; END IF; END;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

```
-- Call the procedure
```

```
CALL check_employee_department(1);
```

result

Employee belongs to another department

Lab 2: Use a FOR LOOP to iterate through employee records and display their names.

```
DELIMITER $$
```

```
CREATE PROCEDURE
```

```
display_employee_names() BEGIN
```

```
    DECLARE done INT DEFAULT 0;
```

```
    DECLARE emp_name
```

```
    VARCHAR(50);
```

```
    DECLARE emp_cursor CURSOR FOR
```

```
        SELECT emp_name FROM employees; -- correct table name
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
    OPEN emp_cursor;
```

```
    read_loop: LOOP
```

```
        FETCH emp_cursor INTO emp_name;
```

```
        IF done = 1 THEN
```

```
            LEAVE
```

```
        read_loop; END IF;
```

```
INSERT INTO employees (emp_name) VALUES (emp_name);

END LOOP;

CLOSE emp_cursor;

SELECT * FROM employees;

TRUNCATE TABLE employees;

END$$

DELIMITER ;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0258 seconds.)

```
CREATE PROCEDURE display_employee_names() BEGIN DECLARE done INT DEFAULT 0; DECLARE emp_name VARCHAR(50); DECLARE emp_cursor CURSOR FOR SELECT
emp_name FROM employees; DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1; OPEN emp_cursor; read_loop: LOOP FETCH emp_cursor INTO emp_name; IF done
= 1 THEN LEAVE read_loop; END IF; INSERT INTO employees(emp_name) VALUES (emp_name); END LOOP; CLOSE emp_cursor; SELECT * FROM employees; TRUNCATE
TABLE employees; END;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

```
CALL display_employee_names();
```

18. SQL Cursors

Theory Questions:

1. What is a cursor in PL/SQL? Explain the difference between implicit and explicit cursors.

Implicit Cursor

- Automatically created by Oracle for **every SQL statement** (like INSERT, UPDATE, DELETE, SELECT INTO).
- Managed by Oracle; you **don't need to declare, open, fetch, or close** it.
- Used when the SQL statement returns **a single row**.

Explicit Cursor

- **Manually declared and controlled** by the programmer.
- Used for **queries that return multiple rows**.
- Requires steps: **DECLARE → OPEN → FETCH → CLOSE**

2. When would you use an explicit cursor over an implicit one?

You would use an **explicit cursor** when you need to handle a query that returns **more than one row** and you want **detailed control** over how each row is processed. Explicit cursors let you **open, fetch, and close** the

cursor manually, making it easier to loop through rows one by one. They are useful when you need to perform calculations, apply conditions, or execute logic for each row of the result set.

In contrast, implicit cursors are automatically created by Oracle for simple SQL statements and are suitable only when the statement returns **a single row** or when no row-by-row processing is needed.

LAB EXERCISES:

Lab 1: Write a PL/SQL block using an explicit cursor to retrieve and display employee details.

DELIMITER //

```
CREATE PROCEDURE display_employee_details() BEGIN
    DECLARE v_emp_id INT;
    DECLARE v_emp_name VARCHAR(50);
    DECLARE v_dept_id INT;
    DECLARE v_salary INT;
    DECLARE v_last_modified DATETIME;
    DECLARE done INT DEFAULT 0;
    DECLARE emp_cursor CURSOR FOR
        SELECT emp_id, emp_name, dept_id, salary, last_modified FROM employees;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN emp_cursor;

    read_loop: LOOP
        FETCH emp_cursor INTO v_emp_id, v_emp_name, v_dept_id, v_salary, v_last_modified; IF done
        THEN
            LEAVE read_loop;
        END IF;
        SELECT CONCAT('Employee ID: ', v_emp_id, ',
            Name: ', v_emp_name,
            ', Dept: ', v_dept_id, ',
            Salary: ', v_salary,
            ', Last Modified: ', v_last_modified) AS employee_details; END
    LOOP;
```

Introduction to DBMS

```
CLOSE emp_cursor;
```

```
END//
```

```
DELIMITER ;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0277 seconds.)

```
CREATE PROCEDURE display_employee_details() BEGIN DECLARE v_emp_id INT; DECLARE v_emp_name VARCHAR(50); DECLARE v_dept_id INT; DECLARE v_salary INT;
DECLARE v_last_modified DATETIME; DECLARE done INT DEFAULT 0; DECLARE emp_cursor CURSOR FOR SELECT emp_id, emp_name, dept_id, salary, last_modified
FROM employees; DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1; OPEN emp_cursor; read_loop: LOOP FETCH emp_cursor INTO v_emp_id, v_emp_name,
v_dept_id, v_salary, v_last_modified; IF done THEN LEAVE read_loop; END IF; SELECT CONCAT('Employee ID: ', v_emp_id, ', Name: ', v_emp_name, ', Dept:
', v_dept_id, ', Salary: ', v_salary, ', Last Modified: ', v_last_modified) AS employee_details; END LOOP; CLOSE emp_cursor; END;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

```
CALL display_employee_details();
```

Lab 2: Create a cursor to retrieve all courses and display them one by one.

```
DELIMITER //
```

```
CREATE PROCEDURE display_courses()
```

```
BEGIN
```

```
    DECLARE v_course_id INT;
```

```
    DECLARE v_course_name VARCHAR(100); DECLARE
```

```
    v_course_duration VARCHAR(50); DECLARE done INT
```

```
    DEFAULT 0;
```

```
    DECLARE c_cursor CURSOR FOR
```

```
        SELECT course_id, course_name, course_duration FROM courses;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
    OPEN c_cursor;
```

```
    c_loop: LOOP
```

```
        FETCH c_cursor INTO v_course_id, v_course_name, v_course_duration;
```

```
        IF done THEN
```

```
            LEAVE c_loop;
```

```
        END IF;
```

```
        SELECT CONCAT('ID: ', v_course_id,
```



```
        ', Name: ', v_course_name,  
        ', Duration: ', v_course_duration) AS course_details; END  
  
LOOP;  
  
CLOSE c_cursor;  
  
END //  
  
DELIMITER ;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0235 seconds.)

```
CREATE PROCEDURE display_courses() BEGIN DECLARE v_course_id INT; DECLARE v_course_name VARCHAR(100); DECLARE v_course_duration VARCHAR(50); DECLARE  
done INT DEFAULT 0; DECLARE c_cursor CURSOR FOR SELECT course_id, course_name, course_duration FROM courses; DECLARE CONTINUE HANDLER FOR NOT FOUND  
SET done = 1; OPEN c_cursor; c_loop: LOOP FETCH c_cursor INTO v_course_id, v_course_name, v_course_duration; IF done THEN LEAVE c_loop; END IF; SELECT  
CONCAT('ID: ', v_course_id, ', Name: ', v_course_name, ', Duration: ', v_course_duration) AS course_details; END LOOP; CLOSE c_cursor; END;
```

CALL display_courses();

19. Rollback and Commit Savepoint

Theory Questions:

1. Explain the concept of SAVEPOINT in transaction management. How do ROLLBACK and COMMIT interact with savepoints?

A **SAVEPOINT** is a marker inside a transaction that allows you to **rollback part of the transaction**, instead of rolling back the entire transaction. It acts like a checkpoint.

Purpose of SAVEPOINT

- To divide a long transaction into smaller parts.
- To undo only a specific portion if something goes wrong.
- To avoid losing the whole transaction.

How ROLLBACK interacts with SAVEPOINT

ROLLBACK TO savepoint_name

- Undo only the changes made **after** that savepoint
- Everything before the savepoint remains safe
- The transaction continues and can still be committed later

Example:

```
SAVEPOINT sp1;  
-- some queries  
ROLLBACK  
TO sp1;
```

How COMMIT interacts with SAVEPOINT

- COMMIT saves all changes permanently
- Once committed, all savepoints inside the transaction are removed
- You cannot rollback to a savepoint after commit

Example:

```
COMMIT;  
-- all savepoints are cleared
```

Short, exam-perfect summary

- **SAVEPOINT** creates a checkpoint inside a transaction.
- **ROLLBACK TO SAVEPOINT** undoes only the changes after that checkpoint.
- **COMMIT** saves all changes permanently and removes all savepoints.

2. When is it useful to use savepoints in a database transaction?

Savepoints are useful when you want to control large or complex transactions and avoid rolling back the entire transaction. They allow partial rollback and help maintain data consistency.

★ **Savepoints are useful in these situations:**

❑ Large transactions with multiple steps

When a transaction has many operations, you can set savepoints to undo only the part that fails instead of starting over.

❑ Optional operations inside a transaction

If some steps may fail but the rest should continue safely.

❑ Error handling inside a transaction

You can roll back only the portion where the error occurred and continue the rest.

❑ Testing small parts of a transaction

During development or debugging, savepoints help test each section without losing everything.

❑ Complex business logic

For example, inserting into multiple related tables — if one insert fails, you can roll back just that section.

❑ Partial undo requirement

When you need to reverse only a specific part of the transaction and keep the previous operations.

Introduction to DBMS

LAB EXERCISES:

Lab 1: Perform a transaction where you create a savepoint, insert records, then rollback to the savepoint.

START TRANSACTION;

INSERT INTO courses (course_name, course_duration) VALUES

('Python Basics', '3 Months');

SAVEPOINT sp1;

INSERT INTO courses (course_name, course_duration) VALUES

('Data Science', '6 Months');

INSERT INTO courses (course_name, course_duration) VALUES

('Machine Learning', '5 Months');

ROLLBACK TO sp1;

COMMIT;

SELECT COUNT(*) FROM courses;

COUNT(*)

13

Introduction to DBMS

		course_id	course_name	course_duration
☐	 Edit  Copy  Delete	1	web development	8 months
☐	 Edit  Copy  Delete	4	python	7 months
☐	 Edit  Copy  Delete	3	php	5 months
☐	 Edit  Copy  Delete	5	mysql	3 month
☐	 Edit  Copy  Delete	6	html	2 months
☐	 Edit  Copy  Delete	7	java	7 moths
☐	 Edit  Copy  Delete	8	mern	10 moths
☐	 Edit  Copy  Delete	9	css	2 moths
☐	 Edit  Copy  Delete	10	Biology	5 months
☐	 Edit  Copy  Delete	11	Computer Science	8 months
☐	 Edit  Copy  Delete	12	Python Basics	3 Months
☐	 Edit  Copy  Delete	13	Data Science	6 Months
☐	 Edit  Copy  Delete	14	Machine Learning	5 Months

Lab 2: Commit part of a transaction after using a savepoint and then rollback the remaining changes.

START TRANSACTION;

INSERT INTO courses (course_name, course_duration) VALUES

('Python Basics', '3 Months');

SAVEPOINT sp1;

INSERT INTO courses (course_name, course_duration) VALUES

('Data Science', '6 Months'),

('Machine Learning', '5 Months');

ROLLBACK TO sp1;

COMMIT;

SELECT * FROM courses;